# Introduction

This section covers the basic concepts that govern development and maintenance of the specification. The actual specification is contained in the following sections.

The reader should have a solid understanding of the main concepts and operating principles of the CAN bus.

## Core design goals

The core design goals listed below help explain the basic DroneCAN concepts and the motivation behind them.

- **Democratic network** - There should be no master node. All nodes in the network should have the same communication rights so that there is no single point of failure.
- **Nodes can exchange long payloads** - Nodes must be provided with a simple way to exchange large data structures that cannot fit into a single CAN frame (such as GNSS solutions, 3D vectors, etc.). DroneCAN should perform automatic transfer decomposition and reassembly at the protocol level, hiding the related complexity from the application.
- **Support for redundant interfaces and redundant nodes** - This is a common requirement for safety-concerned applications.
- **High throughput, low latency communication** - Applications that are dependent on high-frequency, hard real-time control loops, require a low-latency, high-throughput communication method.
- **Simple logic, low computational requirements** - DroneCAN targets a wide variety of embedded systems, from high-performance embedded on-board computers for intensive data processing (e.g., a high-performance Linux-powered machine) to extremely resource-constrained microcontrollers. The latter imposes severe restrictions on the amount of logic needed to implement the protocol.
- **Common high-level functions should be clearly defined** - DroneCAN defines standard services and messages for common high-level functions, such as network discovery, node configuration, node firmware update, node status monitoring (which naturally grows into a vehicle-wide health monitoring), network-wide time synchronization, dynamic node ID allocation (a.k.a. plug-and-play), etc.
- **Open specification and reference implementations** - The DroneCAN specification is open and freely available; the reference implementations are distributed under the terms of the MIT License.

## Specification update and approval process

The DroneCAN development team is charged with advancing the specification based on input from adopters. This feedback is gathered via the mailing list, which is open to everyone.

The set of standard data definitions is one of the cornerstone concepts of the specification (see *data structure description language* (DSDL)). Within the same major version, the specification can be extended only in the following ways:


### Specification

**1. Introduction**
2. Basic concepts
3. Data structure description language
4.1 CAN bus transport layer
4.2 UDP bus transport layer
4.3 MAVLink bus transport layer
4.4 CANFD bus transport layer
5. Application level conventions
6. Application level functions
7. List of standard data types
8. Hardware design recommendations
dsdl
- ardupilot
- uavcan

**Contents**

- Introduction
  - Core design goals
  - Specification update and approval process
  - Referenced sources



### License

This work is licensed under a Creative Commons Attribution 4.0 International License.
Much of the content of this site is based upon prior work by Pavel Kirienko and the UAVCAN Development Team.

- An existing data type can be modified, as long as the modification doesn't break backward compatibility.
- An existing data type can be declared deprecated.
  - Once declared deprecated, the data type will be maintained for at least two more years. After this period its default data type ID may be reused for an incompatible data type.
  - Deprecation will be announced via the mailing list, and indicated in the form of a comment within the DSDL definition.

Link to the repository containing the set of default DSDL definitions can be found on the contacts page.

## Referenced sources

The DroneCAN specification contains references to the following sources:

- CiA 801 - Application note - Automatic bit rate detection.
- CiA 103 - Intrinsically safe capable physical layer.
- CiA 303 - Recommendation - Part 1: Cabling and connector pin assignment.
- IEEE 754 - Standard for binary floating-point arithmetic.
- ISO 11898-1 - Controller area network (CAN) - Part 1: Data link layer and physical signaling.
- ISO 11898-2 - Controller area network (CAN) - Part 2: High-speed medium access unit.
- ISO/IEC 10646:2014 - Universal Coded Character Set (UCS).
- ISO/IEC 14882:2014(E) - Programming Language C++.
- “Implementing a Distributed High-Resolution Real-Time Clock using the CAN-Bus”, M. Gergeleit and H. Streich.
- “In Search of an Understandable Consensus Algorithm (Extended Version)”, Diego Ongaro and John Ousterhout.